CRISIS GUARD

PROJECT REPORT

SUBMITTED BY

FEBIN BABU (P6AXMCA008)

FOR THE AWARD OF THE DEGREE

OF MASTER OF COMPUTER APPLICATION (MCA)

IN COMPUTER SCIENCE

UNIVERSITY OF CALICUT

DEPARTMENT OF COMPUTER SCIENCE

CALICUT UNIVERSITY OF REGIONAL CENTER, PERAMBRA

2023-2025

UNIVERSITY OF CALICUT

CERTIFICATE

This is to certify that the project report entitled **CRISIS GUARD** submitted by **FEBIN BABU (P6AXMCA008),**to University of Calicut for the award of the degree of MCA Computer Science is a bona fide record of the project work carried out by their under my supervision and guidance. The content of the report, in full or parts have not been submitted to any other Institute or University for the award of any other degree or diploma.

AJANYA CK	AMAL HUDA
ASSISTANT PROFESSOR	ASSOCIATE COORDINATOR

Certified that the candidate was examined by us in the Project Viva Voce Examination held on............................ and his Register Number is P6AXMCA008.

Examiners:

1.

2.

DECLARATION

I, FEBIN BABU hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person or material which has been accepted for the award of any other degree of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

DATE: SIGNATURE:

NAME: FEBIN BABU

Reg No: P6AXMCA006

ACKNOWLEDGEMENT

The success of the projects depends upon the effort invested it’s my duty to acknowledge and thank to individuals who have the contributed in the successful completion of the projects. first and for most, I praise the god almighty for the grace he Showered one me during my studies as well as day life activities.

I wish to express my heartfelt gratitude to **Mrs. Amal Huda Associate Coordinator** , Department of Computer Science for The Encouragement Inspiring Guidance throughout the Preparation of Project.

I am very much thankful to **smt. Ajanya. C K Assistant professor** of Calicut University Regional Centre Perambra, For her Valuable Guidance to This Project Successfully.

I take This opportunity to express my profound and great thanks to our teachers **Mrs. Greeshma K K, Mrs. Nileena Vijayan,** Teachers of Department of Computer Science Calicut University Regional Centre, Perambra for Their Support to complete my project.

I wish to express my love and respect for my parents, for their support contribution and encouragement which helped me a lot to complete this project successfully.

I finally give a bunch of thanks to my friends and all my well-wishes who had supported me directly and indirectly throughout the project with their constructive criticisms.

DATE: **FEBIN BABU**

CONTENT

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

ABSTRACT

Crisis Guard is an innovative disaster management system designed to mitigate the impact of natural disasters on communities worldwide. This cutting-edge platform integrates advanced weather prediction, emergency response, and relief operations to enable prompt and effective disaster management. By identifying disaster-prone areas, facilitating communication and coordination among stakeholders, and streamlining response mechanisms, Crisis Guard reduces the loss of life and property, enhances early recovery, and ensures a unified response to disasters.

The system's scalability, flexibility, and user-friendly interface make it an indispensable tool for governments, emergency responders, and communities. By interfacing with various technologies, including weather prediction systems, emergency response systems, GIS mapping tools, and historical disaster dataset repositories, Crisis Guard provides a comprehensive and integrated approach to disaster management. This project has the potential to revolutionize disaster management practices globally, saving countless lives and reducing economic and environmental impacts. With its advanced features and technologies, Crisis Guard is poised to become a critical component of disaster management efforts worldwide, helping to create a safer and more resilient future for all.

CHAPTER 2

INTRODUCTION

Natural disasters have become an unfortunate reality in today's world. The frequency and severity of these events are increasing, resulting in devastating losses of life, property, and infrastructure. The importance of effective disaster management cannot be overstated.

Traditional disaster management approaches often focus on response and relief efforts. However, this reactive approach can be inefficient, leading to delayed responses, inadequate resource allocation, and increased loss of life and property. A proactive and comprehensive approach to disaster management is essential. This involves identifying disaster-prone areas, developing targeted preparedness and response strategies, and ensuring effective communication and coordination among all stakeholders.

Crisis Guard is an innovative, self-contained disaster management system designed to revolutionize the way we prepare for, respond to, and recover from disasters. By integrating advanced weather prediction, emergency response, and relief operations, Crisis Guard enables prompt and effective disaster management. This project aims to significantly reduce the loss of life and property, enhance early recovery, and ensure a prompt and effective response to various disasters through well-coordinated efforts and advanced systems.

Crisis Guard offers a range of features that make it an indispensable tool for disaster management. The system's ability to identify disaster-prone areas enables targeted preparedness and response efforts. Effective communication and coordination among response teams, authorities, and communities are facilitated through the system, ensuring a unified response to disasters. Crisis Guard also streamlines response mechanisms for efficient disaster management, helping to plan the movement of resources within specific time frames and ensuring timely deployment. Furthermore, the system encourages community participation in disaster preparedness, fostering a collaborative approach, and mobilizes and trains disaster volunteers, amplifying response efforts.

Crisis Guard is designed to be scalable, flexible, and user-friendly. The system will interface with various technologies, including weather prediction systems, emergency response systems, GIS mapping tools, mobile devices, web browsers, and historical disaster dataset repositories. By integrating these technologies, Crisis Guard provides a comprehensive and integrated approach to disaster management.Crisis Guard is a groundbreaking disaster management system that has the potential to save countless lives and reduce economic and environmental impacts. By providing a comprehensive and integrated approach to disaster management, Crisis Guard is an essential tool for governments, emergency responders, and communities around the world.

CHAPTER 3

OVERALL DESCRIPTION

3.1 Product Perspective

Context and Origin

CRISIS GUARD is an innovative, self-contained software solution designed to revolutionize disaster management by integrating weather prediction, emergency response coordination, and relief operations. Its primary goal is to minimize the loss of life and property during disasters by enhancing preparedness and response capabilities.

Relationship to Existing Systems

CRISIS GUARD modernizes disaster management by replacing manual and fragmented processes with an advanced, technology-driven approach. It integrates seamlessly with:

- National Weather Service (NWS) for real-time weather forecasting and alerts.
- Emergency Response Systems (ERS) for effective alert dissemination and response coordination.
- Geographic Information Systems (GIS) for accurate mapping and spatial analysis.

Component of a Larger System

As a pivotal element of the national disaster management framework, CRISIS GUARD interacts with various entities, including:

- National Emergency Management Agency (NEMA)
- State and Local Emergency Operations Centers (EOCs)
- Relief organizations and NGOs

3.2 Interfaces

CRISIS GUARD provides a range of interfaces to facilitate efficient data exchange and user interaction, including:

- Weather prediction systems via APIs.
- Emergency response systems via APIs.
- GIS mapping tools via APIs.

- User-friendly interfaces for:
    - Mobile devices (iOS and Android)
    - Web browsers (Chrome, Firefox, and Safari)
    - Historical disaster dataset repository for data import and analysis.

3.3 External Interfaces

CRISIS GUARD supports robust external interactions, such as:

- API integrations with weather forecasting and emergency response systems.
- Data exchange capabilities with GIS mapping tools.
- Intuitive user interfaces for mobile devices and web browsers.
- Import functionality for historical disaster datasets to enhance predictive analytics.

Assumptions and Dependencies

To ensure the successful operation of CRISIS GUARD, the following assumptions and dependencies are considered:

- Reliable internet connectivity.
- Access to accurate and timely weather forecasting data.
- Active collaboration with emergency response agencies and organizations.
- Availability of comprehensive historical disaster datasets for analysis and prediction.

3.4 Product Features:

Disaster-Prone Area Identification: Identifies locations prone to disasters, enabling targeted preparedness and response efforts.

• Communication and Coordination: Facilitates communication and coordination among response teams, authorities, and communities.

• Response Mechanisms: Streamlines response mechanisms for efficient disaster management.

• Resource Management: Helps plan the movement of resources within specific time frames, ensuring timely deployment.

• Community Involvement: Encourages community participation in disaster preparedness, fostering a collaborative approach.

• Volunteer Management: Mobilizes and trains disaster volunteers, amplifying response efforts.

• Evacuation and Search and Rescue: Facilitates evacuation, search, and rescue operations, saving lives and reducing casualties.

• Victim Identification: Assists in identifying deceased individuals, providing closure for families and facilitating official procedures.

• Advanced Victim Detection: Utilizes technology to locate victims in rubble or mud, expediting rescue efforts

CHAPTER 4

REQUIREMENT ANALYSIS&SPECIFICATION

A system study is a process of studying a procedure to identify the objectives and purposes of a system as well as to analyse the existing system's problems and drawbacks. The system analysis will create systems and procedures that will achieve these objectives as well as solutions to the problems in an efficient way. It is the process of gathering and interpreting facts, diagnosing problems and using the information to recommended improvements to the system. Training, experience and common sense are required for the collection of the information needed to do the analysis.

4.1 PROPOSED SYSTEM

The primary goal of disaster management is to reduce the potential loss of life and property damage, as well as to ensure a quick and effective response and recovery. This system helps to organization, planning and application of measures preparing, responding to initial recovery from disasters. Weather prediction plays a vital role in disaster management, as it provides critical information to anticipate and respond to weather-related disasters such as hurricanes, floods, wildfires, and extreme heat events.

4.2USER REQUIREMENT SPECIFICATION

The system after careful analysis has been identified to be presented with the following module:

- ADMIN
- COORDINATOR
- PUBLIC
- VOLUNTEER
- EMERGENCY RESPONSE TEAM

ADMIN

- Add and manage camp
- Add and manage camp coordinators
- Complaint manage
- Guidelines to coordinator
- Notification
- Change password
- Verify emergency Response Team

COORDINATOR

- Stock management
- Member registration
- Check needs
- Medical support
- Change password

PUBLIC

- View needs
- Registration
- Donate goods

- Send emergency request
- Weather prediction
- Change password

VOLUNTEER

- View needs
- Collect goods
- Collection management
- Manage services
- Request for medical support
- Change password

EMERGENCY RESPONSE TEAM

- Registration
- Login
- View emergency requests
- Weather prediction
- Change password

4.3SOFTWARE REQUIREMENT SPECIFICATION

Software selection is an important work in a project development cycle. Software must be selected in accordance with the application and latest technology available. Main software used:

PyCharm: PyCharm provides smart code completion, code inspections, onthefly error highlighting and quick-fixes, along with automated code refactoring's and rich navigation capabilities.

4.4 FEASIBILITY STUDY

Feasibility is defined as the practical extent to which a project can be performed successfully. To evaluate the feasibility, a feasibility study performed, which determine whether the solution considered to accomplish the requirements is practical and workable in the software. Information such as resource availability, cost estimation for software development, benefits of the software to the organization after it is developed and cost to be incurred on its maintenance are considered during the feasibility study. The objective of the feasibility study is to establish the reason for developing the software that is acceptable to users, adaptable to change and confirm-able to established standards. The result should inform the decision of whether to go ahead with a more detailed analysis.

The feasibility study is done in these phases:

- Technical feasibility
- Economical feasibility
- Operational feasibility

TECHNICAL FEASIBILITY

It says that the system must be evaluated from the technical point of view. Having identified an outline system, the investigation must go on to suggest the type of equipment, required method for developing the system, method of running the system once it has been designed. The current system developed is technically feasible thus it provides easy access to the user.

EC0NOMICAL FEASIBILITY

Economic feasibility is assessed based on total expenditure and profit analysis. There are certain queries needed to be answered if the enterprise or company can develop the system, Is it profitable enough to stand above the expenses for its development of the system.

OPERATIONAL FEASIBILITY

The purpose of the operational feasibility study is to determine whether the new system will be used if it is developed and implemented from users that will undermine the possible application benefits.

4.5 SYSTEM SPECIFICATION

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product.

HARDWARE AND SOFTWARE REQUIREMENTS

The following requirements are only the minimal requirements to run this utility more successfully and efficiently, there should be sufficient memory and software tools for efficient processing**.**

Hardware Requirements

- **Android Device**

CPU : Intel Pentium and above
GPU : Any GPU with at least 200MHz speed
RAM : 4GB or more

Display : LED display

Camera : 2MP or above

Keyboard : windows compatible

Mouse: Windows compatible

Software Requirements

One of the most difficult task is selecting software for the system, once the system requirements is found out then we have to determine whether a particular software package fits for those system requirements.

- Database : MySQL
- Operating System : Windows 8 or above
- Front End : HTML,CSS,BOOTSTRAP, JAVASCRIPT,DART
- Back End : Python-Django, Flutter/Java
- IDE : PyCharm community / Android Studio / Eclipse
- Web browser : Chrome, Explorer, Edge…etc.

4.6 FUNDAMENTAL DESIGN CONCEPTS

The design concepts provide the software designer with a foundation from which more sophisticated methods can be applied.

1. Abstraction: Abstraction is the process or result of generalization by reducing the information content of a concept or an observable phenomenon.

2. Refinement: It is the process of elaboration. A hierarchy is developed by decomposing a macroscopic statement of function in a step-wise fashion until programming language statements are reached.

3. Modularity: Software architecture is divided into components called modules.

4. Software Architecture: It refers to the overall structure of the software and the ways in which that structure provides conceptual integrity for a system. Good software architecture will yield a good return on investment with respect to the desired outcome of the project, .

5. Control Hierarchy: A program structure that represents the organization of a program component and implies a hierarchy of control.

6. Structural Partitioning: The program structure can be divided both horizontally and vertically. Horizontal partitions define separate branches of modular hierarchy for each major program function.

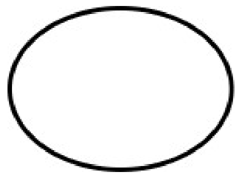
7. Data Structure: It is a representation of the logical relationship among individual elements of data.

8. Software Procedure: It focuses on the processing of each module individually.

9. Information Hiding: Modules should be specified and designed so that information contained within a module is inaccessible to other modules that have no need for such information.

4.7.1DESIGN NOTATIONS

Process



A process shows a transformation or manipulation of data flows within the system. A process transforms incoming data flow into outgoing data flow.

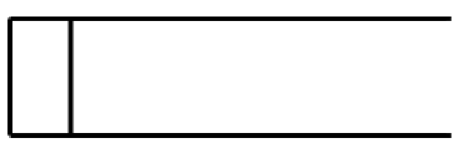
External Entity

External entities are outside the system, but they either supply input data into the system or use system output. External entities are represented by a rectangle.

Data Flows

A data flow shows flow of information from source to destination. A data flow is represented by a line, with arrowhead showing the direction of flow.

Data Source



A repository of data that is to be stored for use by one or more processes; may be as simple as a buffer or queue or as sophisticated as a relational database.

4.7.2 Data Flow Diagram

A graphical representation is used to describe and analyse the movement of data through a system manual or automated including the processes, storing of data and delays in the system. Data flow diagrams are the central tool and the basis from which other components are developed. The transformation of the data from input to output through process may be described logically and independently of the physical components associated with the system. They are termed logical data flow diagrams, showing the actual implementation and the movement of data between people, departments and workstations. DFD is one of the most important modelling tools used in physical design. DFD shows the flow of data through different process in the system.

4.8 DESIGN PROCESS

A table is a data structure that organizes information into rows and columns. It can be used to both store and display data in a structured format. Databases often contain multiple tables, with each one designed for a specific purpose. Each table may include its own set of fields, based on what data the table needs to store. In database tables, each field is considered a column, while each entry (or record), is considered a row. A specific value can be accessed from the table by requesting data from an individual column and row. There are primary key fields for uniquely identifying a record in a table.

4.8.1 Database Design

Database design is one of the most important aspects of Web programming. Successful implementation of any system with Data Storage will always need proper database design. To create a great database design, you will not only have to master the Database technology but will have to master the database design process and normalization.

4.8.2 Normalization

Database Normalization is a technique of organizing the data in the database. Normalization is a systematic approach of decomposing tables to eliminate data redundancy(repetition) and undesirable characteristics like Insertion, Update and Deletion Anomalies. It is a multi-step process that puts data into tabular form, removing duplicated data from the relation tables. In our design the tables have been normalized up to third normalization form. Normalization applied during database design are:

- First Normal Form (1NF)
- Second Normal Form (2 NF)

First Normal Form

If a relation contains composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute. A relation is in first normal form if every attribute in that relation is singled valued attribute.

Our application satisfies 1 NF. The first normal form states that:

• Every column in the table must be unique.

• Separate tables must be created for each set of related data.

• Each table must be identified with a unique column or concatenated columns called the primary key.

• No rows may be duplicated.

• no row/column intersections contain a null value.

Second Normal Form

A relation is said to be in 2NF if and only if it satisfies all 1NF conditions for the primary key and every non-primary key attributes of the relation is fully dependent on its primary key alone. If a non-key attribute is not dependent on the key, it should be removed from the relation and places as a separate relation. i.e., the fields of the table in 2NF are all related to primary key.

Input Design

The input design is the process of converting the user-oriented inputs into the computer-based format. The goal of designing input data is make the automation as easy and free from errors as possible. For providing a good input design for the application, easy data input and selection features are adopted.

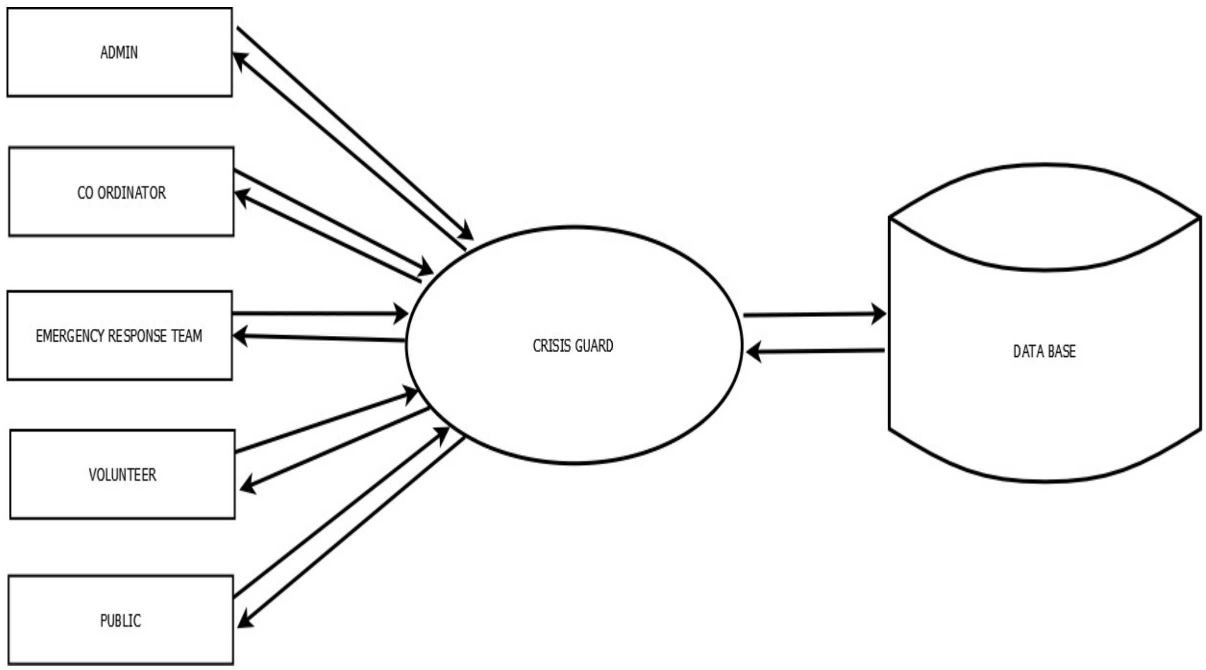
Output Design

The output design has been done so that the results of processing should be communicated to the user. Effective output design will improve the clarity and performance of outputs. Output is the main reason for developing the system and the basis on which they will evaluate the usefulness of the application. Output design phase of the system is concerned with the convergence of information to the end user-friendly manner. The output design should be efficient, intelligible so that system relationship with the end user is improved and they're by enhancing the process of decision-making.

4.9 ARCHITECTURE DIAGRAM

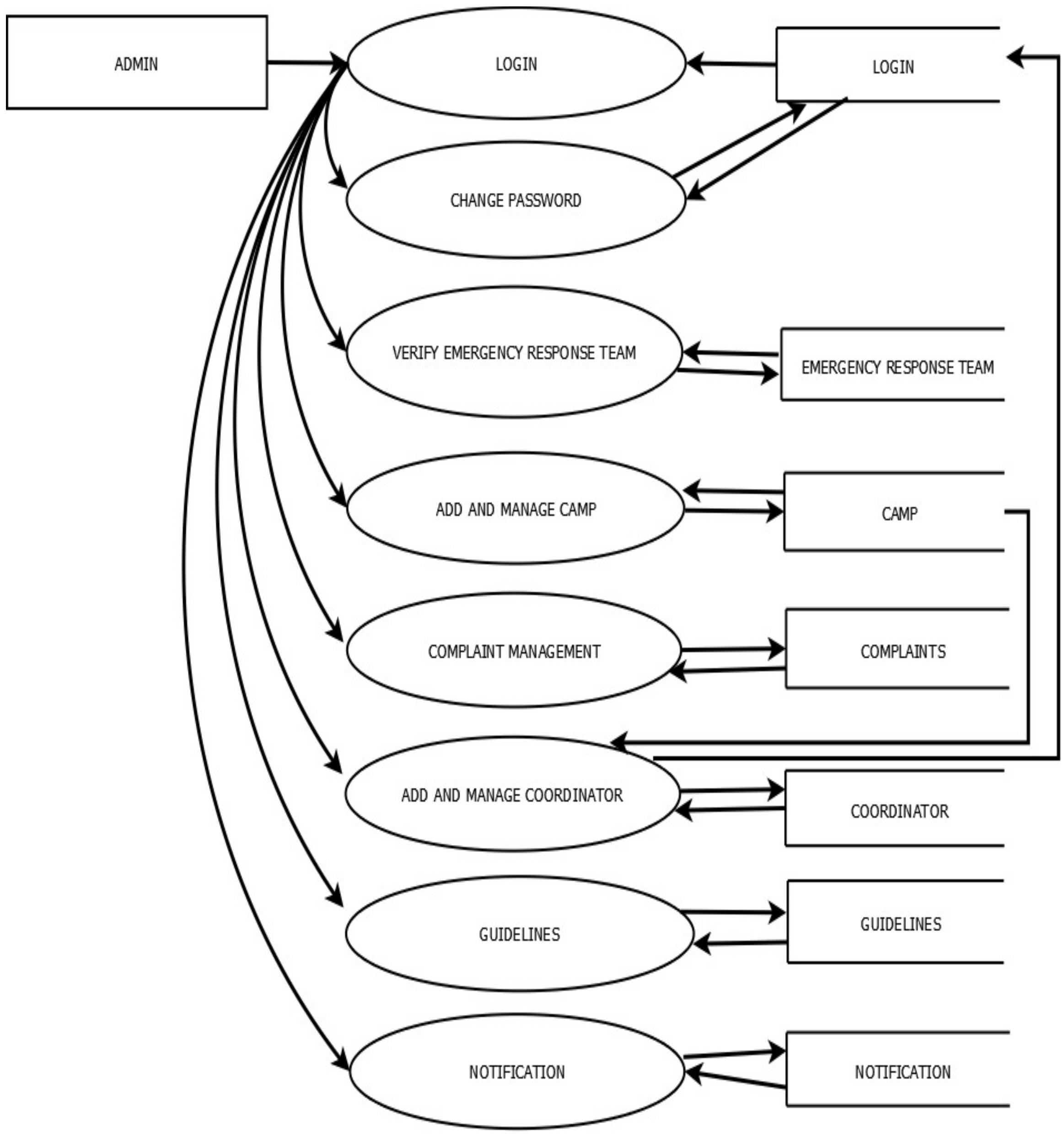
a) DFD (DATA FLOW DIAGRAM)

LEVEL 0



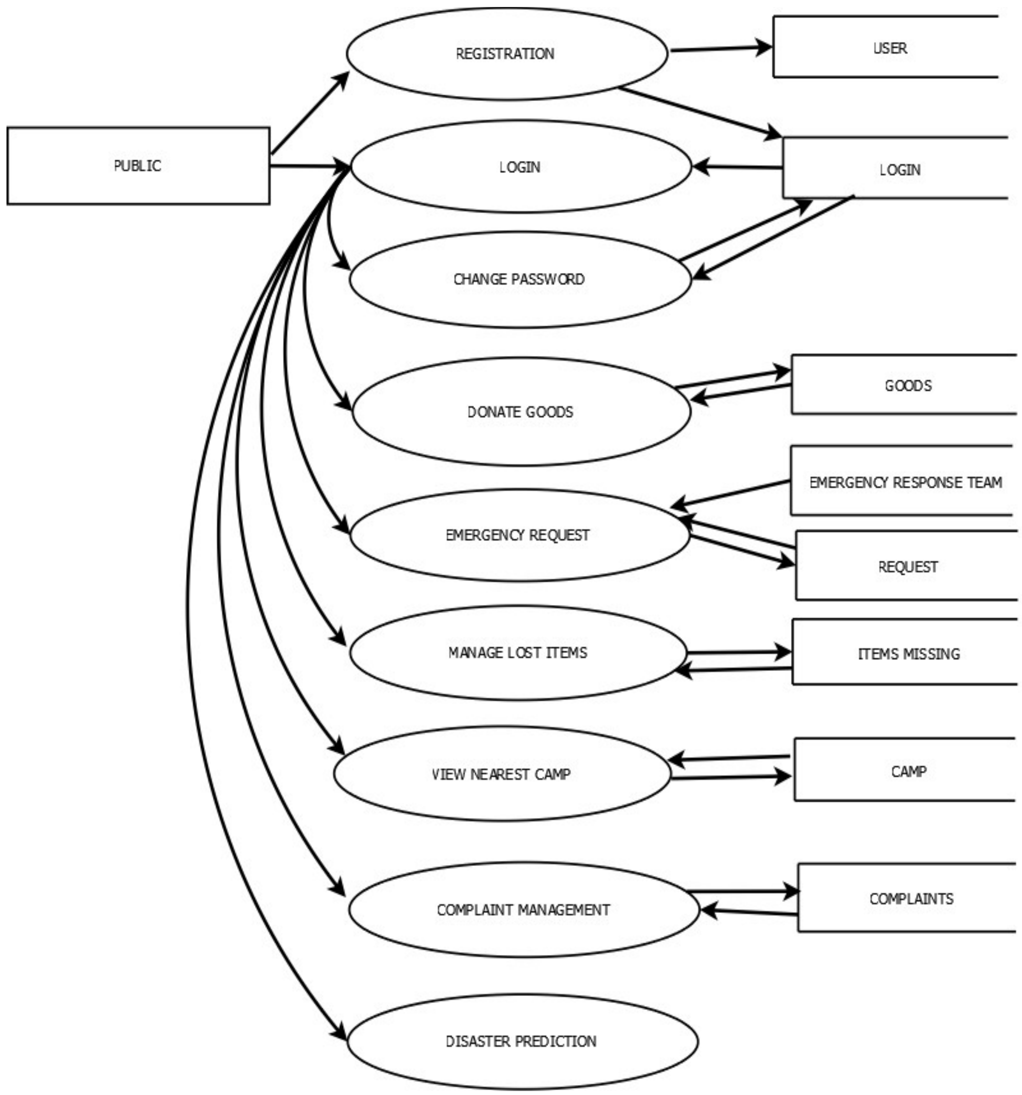
(figure1 = LEVEL 0)

LEVEL 1.0: ADMIN



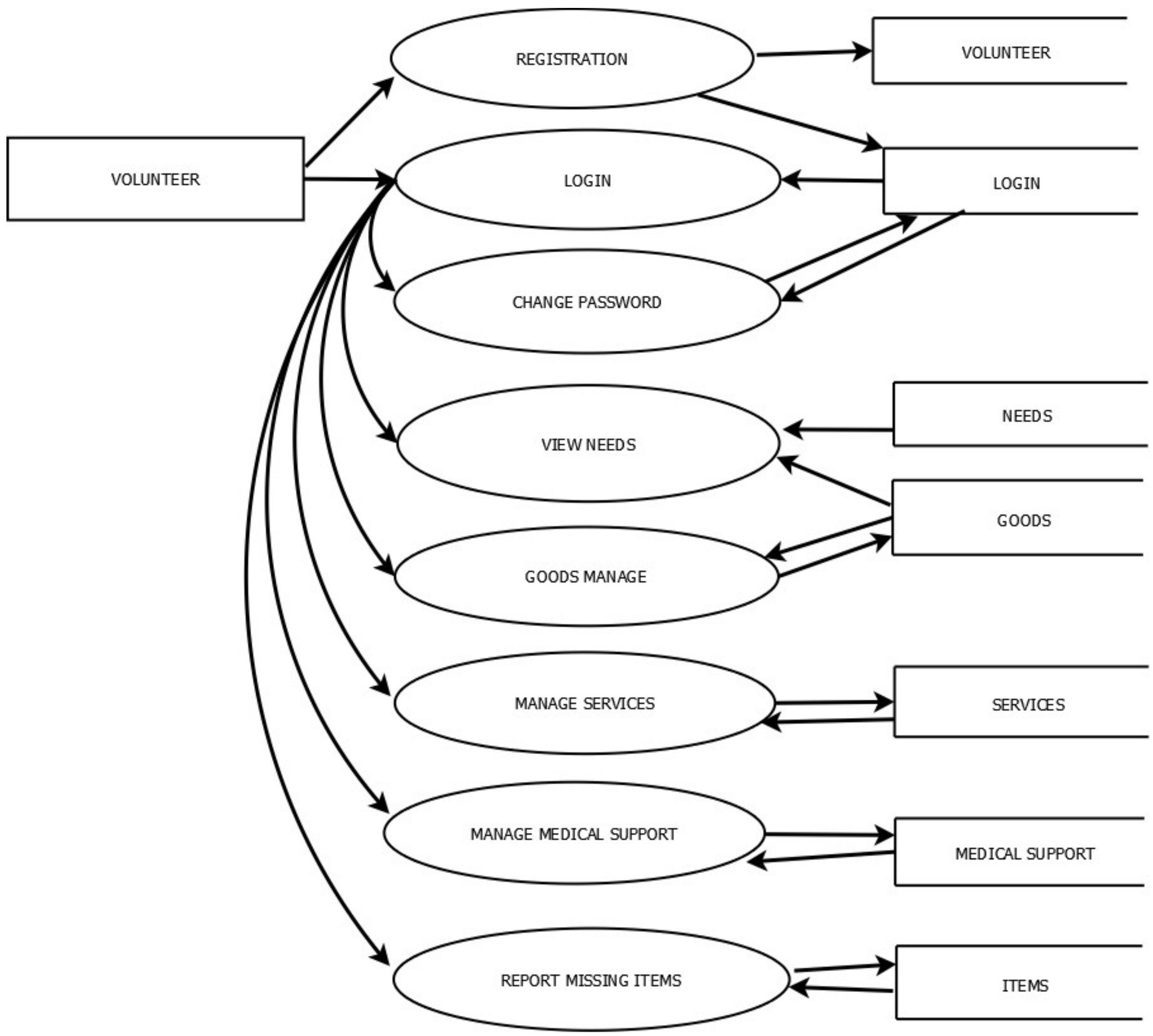
(figure2 LEVEL 1.0 ADMIN)

LEVEL 1.1: PUBLIC



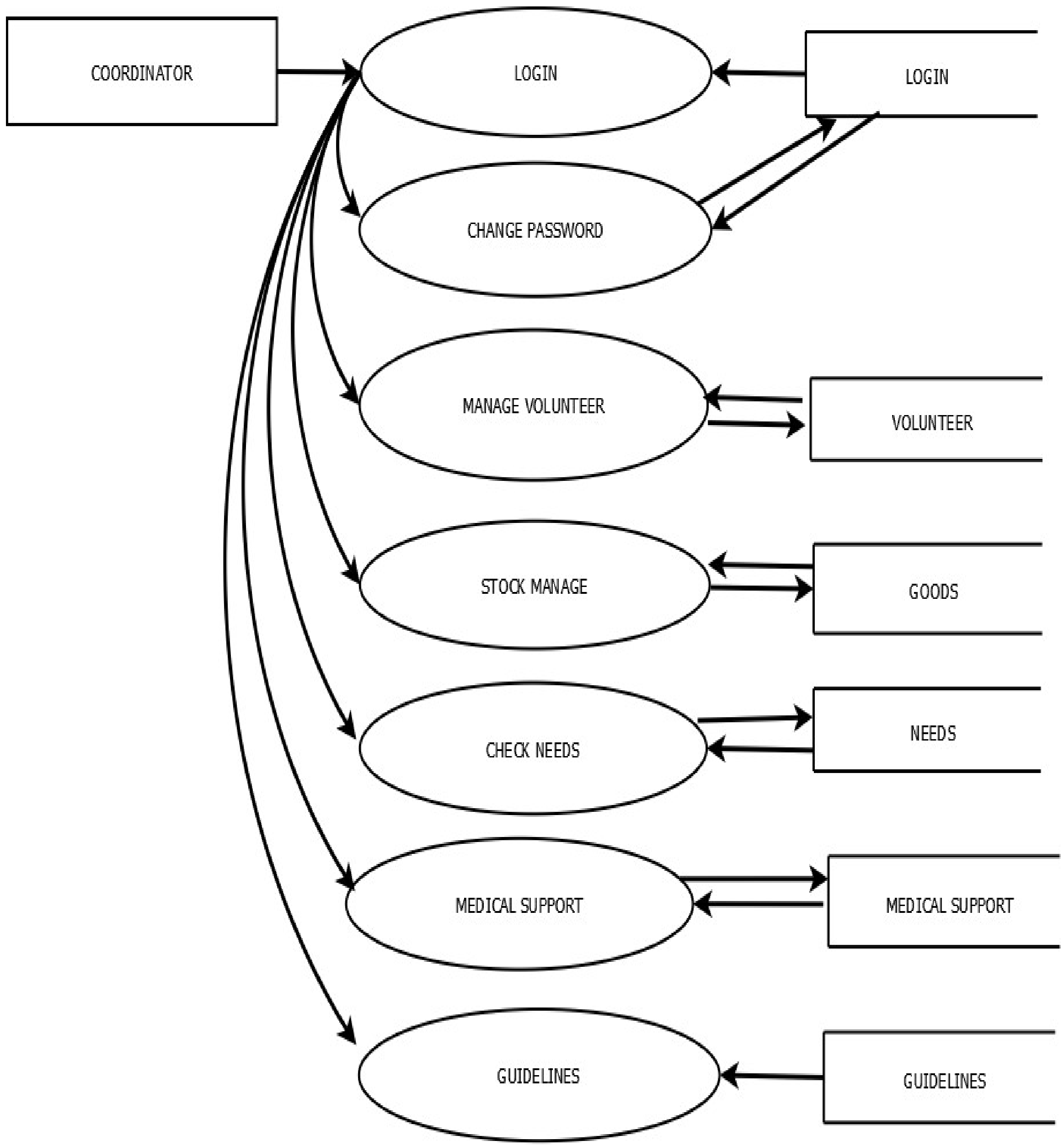
(figure3 = LEVEL 1.1 PUBLIC)

LEVEL 1.2: VOLUNTEER



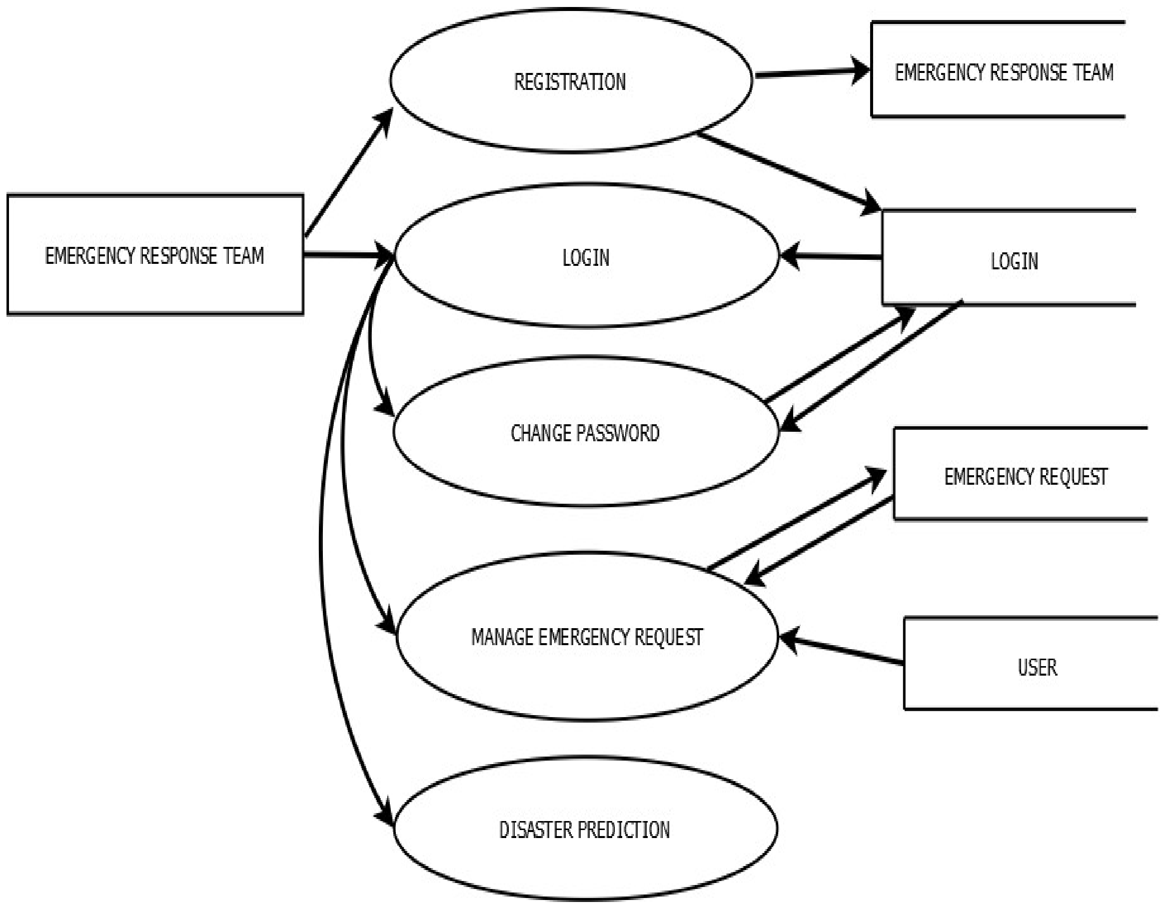
(figure4= LEVEL=1.4 VOLUNTEER)

LEVEL 1.3: COORDINATOR



(figure 5 LEVEL=1.3 COORDINATOR)

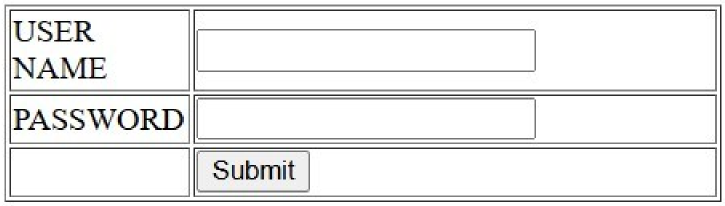
LEVEL 1.4 EMERGENCY RESPONSE TEAM



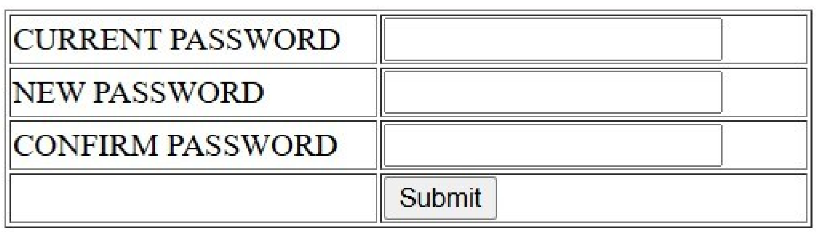
(figure 6 LEVEL=1.4 EMERGENCY RESPONSE TEAM)

USER INTERFACE LAYOUT

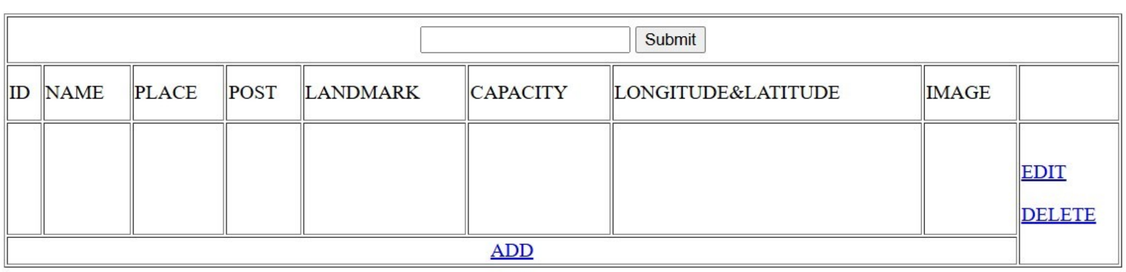
b) FORMS:



(figure 7= login page)



(figure 8= change password)



(figure 9= camp registration)

NAME	
IMAGE	
DATE OF BIRTH	
GENDER	○ MALE ○ FEMALE
PLACE	
POST	
PIN	
PHONE NUMBER	
DESIGNATION	
EXPERIENCE	
EMAIL	
USERNAME	
PASSWORD	
	Submit

(figure10 = registration)

REQUEST TYPE	DETAILS	DATE	STATUS	
				EDIT DELETE

(figure11 = emergency request)

TYPE	
NAME	
DETAILS	
DATE	
STOCK	
QUANTITY	
PLACE	
ADD	

(figure12 = stock manage)

NOTIFICATION	DETAILS	DATE	EDIT DELETE
ADD			

(figure 13= notification)

NAME		SEARCH				
NAME	**IMAGE**	**CAMP DETAIL**	**PHONE NUMBER**	**DOB**	**EMAIL**	**EDIT** **DELETE**
ADD						

(figure14 = view coordinator)

c) TABLE:

LOGIN:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
username	varchar	50	Not null
password	varchar	50	Not null
type	varchar	50	Not null

(table1 = login)

USER:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
first_name	varchar	50	Not null
last_name	varchar	50	Not null
gender	varchar	30	Not null
age	int	20	Not null
address	varchar	100	Not null
phone_number	int	20	Not null
LOGIN_id	int	20	Foreign key
e-mail	varchar	50	Not null

(table 2 = user)

COORDINATOR:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
name	varchar	50	Not null
address	varchar	100	Not null
dob	varchar	20	Not null
gender	varchar	20	Not null
phone_number	int	20	Not null
e-mail	varchar	50	Not null
image	varchar	300	Not null
CAMP_id	int	20	Foreign key
LOGIN_id	int	20	Foreign key

(table 3 = coordinator)

VOLUNTEER:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	Int	20	Primary key
name	varchar	50	Not null
image	varchar	300	Not null
dob	varchar	20	Not null
gender	varchar	20	Not null
place	varchar	50	Not null
post	int	20	Not null
phone	int	20	Not null
e-mail	varchar	50	Not null
LOGIN_id	Int	20	Foreign key

(table 4 = volunteer)

EMERGENCY RESPONSE TEAM:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
name	varchar	50	Not null
place	varchar	50	Not null
post	varchar	50	Not null
phone	int	20	Not null
Experience	varchar	100	Not null
LOGIN_id	int	20	Foreign key

(table 5 = ert)

COMPLAINT TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
complaints	varchar	100	Not null
replay	varchar	100	Not null
date	varchar	20	Not null
USER_id	int	20	Foreign key
COORDINATOR_id	int	20	Foreign key

(table 6 = complaint)

SERVICE TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
type	varchar	50	Not null
details	varchar	50	Not null
VOLUNTEER_id	int	20	Foreign key

(table 7 = service**)**

DONATE TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
date	varchar	50	Not null
quantity	varchar	50	Not null
NEEDS_id	int	20	Foreign key
USER_id	int	20	Foreign key

(table 8= donate)

GOODS TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
type	varchar	50	Not null
name	varchar	50	Not null
date	varchar	50	Not null
details	varchar	50	Not null
stock	varchar	50	Not null
quantity	varchar	50	Not null
place	varchar	50	Not null
COORDINATOR_id	int	20	Foreign key

(table 9= goods)

NOTIFICATION TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
notification	varchar	50	Not null
date	varchar	50	Not null
details	varchar	50	Not null

(table 10 = notification)

GUIDELINE TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
guidelines	varchar	50	Not null
details	varchar	50	Not null
date	varchar	50	Not null
COORDINATOR_id	int	20	Foreign key
type	varchar	50	Not null

(table 11 = guideline)

ITEM TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
type	varchar	50	Not null
details	varchar	50	Not null
picture	varchar	50	Not null
status	varchar	50	Not null
LOGIN_id	int	20	Foreign key

(table 12 = item)

MEDICAL SUPPORT TABLE:

COLUMN NAME	DATA TYPE	LENGTH	CONSTRAINT
id	int	20	Primary key
details	varchar	50	Not null
date	varchar	50	Not null
status	varchar	50	Not null
COORDINATOR_id	int	20	Foreign key

(table 13 = medical)

CHAPTER 5

IMPLEMENTATION

5.1 SYSTEM IMPLEMENTATION

Implementation is a process of ensuring that the information system is operational. It involves – Constructing a new system from scratch Constructing a new system from the existing one. Implementation allows the users to take over its operation for use and evaluation. It involves training the users to handle the system and plan for a smooth conversion. System maintenance is an ongoing activity, which covers a wide variety of activities, including removing program and design errors, updating documentation and test data and updating user support. During the design phase, the products structure, its undergoing data structures, the general algorithms and the interfaces and control/data linkages needed to support communication among the various sub structures were established. Implementation process is simply a translation of the design abstraction into the physical realization, using the language of the target architecture.

There are three types of implementation:

- Implementation of a computer system to replace a manual system.
- Implementation of a new computer system to replace an existing one.
- Implementation of a modified application to replace an existing one computer.

5.1.1 Implementation Procedures

Implementation is a process of ensuring that the information system is operational. The common approaches for implementation are:

PARALLEL CONVERSION

In parallel conversion the existing system and new system operate simultaneously until the project team is confident that the new system is working properly. The outputs from the old system continue to be distributed until the new system has proved satisfactorily parallel conversion is a costly method because of the amount of duplication involved.

DIRECT CONVERSION

Under direct conversion method the old system is discontinued altogether and the new system becomes operational immediately. A greater risk is associated with direct conversion is no backup in the in the case of system fails.

PILOT CONVERSION

A pilot conversion would involve the changing over of the part of the system either in parallel or directly. Use of the variation of the two main methods is possible when part of the system can be treated as a separate entity.

5.2 SYSTEM MAINTENANCE

The maintenance is an important activity in the life cycle of a software product. Maintenance Includes all the activities after the installation of software that is performed to keep the System operational. The maintenance phase of a software life cycle is the time period in Which a product performs useful work. Maintenance is classified into four types:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance
- Preventive Maintenance

5.2.1 CORRECTIVE MAINTENANCE

Corrective maintenance refers to changes made to repair defects in the design, coding, or Implementation of the system. Corrective maintenance is often needed for repairing Processing or performance failures or making changes because of previously uncorrected Problems or false assumptions. Most corrective maintenance problems surface soon after the Installation.

5.2.2 ADAPTIVE MAINTENANCE

Adaptive maintenance involves making changes to an information system to evolve its Functionality or to migrate it to different operating environment. Adaptive maintenance is Usually less urgent than corrective maintenance.

5.2.3 PERFECTIVE MAINTENANCE

Perfective maintenance involves making enhancements to improve processing performance, Interface usability, or to add desired, but not necessarily required, system features. Many System professionals feel that perfective maintenance is not really the maintenance but new Development.

5.2.4 PREVENTIVE MAINTENANCE

Preventive maintenance is the only maintenance activity which is carried out without formal Maintenance request from the user. When a software company or maintenance agency Realizes that the methodologies used in a program have become obsolete, it may decide to Develop or modify parts of the program, which do not confirm to the current trends. Of these Types, more time and money is spending on perfective than on corrective and adaptive Maintenance together.

CHAPTER 6

TESTING

6.1 TESTING

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform allocated functions. Testing is the final verification and validation activity within the organization itself. Testing is done to achieve the following goals: to test the quality of the product, to find and eliminate any residual errors from previous stages, to validate the software as a solution to the original problem, to demonstrate the presence of all specified functionality in the product, to estimate the operational reliability of the system. During testing the major activities are concentrated on the examination and modification of the source code.

6.1.1 Testing Methodology

MANUAL TESTING

Manual testing is a software testing process in which test cases are executed manually without using any automated tool. All test cases executed by the tester manually according to the end user's perspective. It ensures whether the application is working, as mentioned in the requirement document or not. Test cases are planned and implemented to complete almost 100 percent of the software application. Test case reports are also generated manually. Manual Testing is one of the most fundamental testing processes as it can find both visible and hidden defects of the software. The difference between expected output and output, given by the software, is defined as a defect. The developer fixed the defects and handed it to the tester for retesting.

Manual testing is mandatory for every newly developed software before automated testing. This testing requires great efforts and time, but it gives the surety of bug free software.

6.1.2 Different Testing

UNIT TESTING

It is the first level of testing. Each module is tested individually and focus is given for finding errors limited to each individual module and correcting them. The different modules of the system are tested individually and corrected all errors. Each module is focused to work satisfactorily with regard to the expected output from the module. In computer programming, unit testing is a software testing method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use.

INTEGRATION TESTING

Integration testing is a systematic testing for constructing the program structure while at the same time conducting tests to uncover errors associated with in the interfaces; all the modules are combined and tested as a whole. Integration testing tests whether the system works correctly with units that are integrated to do a particular function.

• Top-Down Integration

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control 51 hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner.

• Bottom-Up Integration

This method begins the construction and testing with the modules at the lowest in the program structure. Since the modules are integrated from the bottom up, processing required for modules subordinate to a given level is always available and the need for stubs is eliminated.

VALIDATION TESTING

For each input forms validation testing are done to ensure that only allowed values will be entrees functions for which it is intended and meets the organization's goals and user needs. Validation is done during testing like feature testing, integration testing, system testing, load testing, compatibility testing, stress testing, etc.

Front-End Validation

Everything is validated on the server to prevent someone creating an alternative client to access/manipulate database. Front end is also necessary as it improves efficiency and prevents the server being accessed with inappropriate data. Front end validation is for data entry help and contextual messages. This ensures that the user has a hassle-free data entry experience and minimizes the round-trip required for validating correctness. Front end validation validates all input in a modern application, providing the user with quick feedback a possible issue.

Back-End Validation

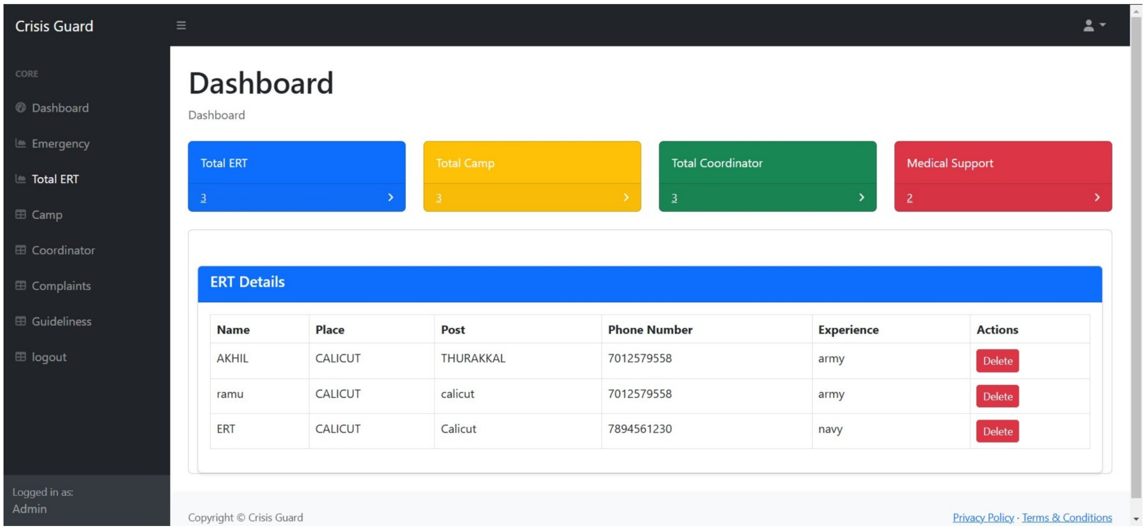
Back-end validation is also an essential part. It has to ensure that the data coming in is needed valid. Additionally, depending on the architecture, generally re-use the middle-tier business logic among multiple components so need to ensure the rules that are applied are always consistent – regardless of 52 what the front-end logic enforces.

OUTPUT TESTING

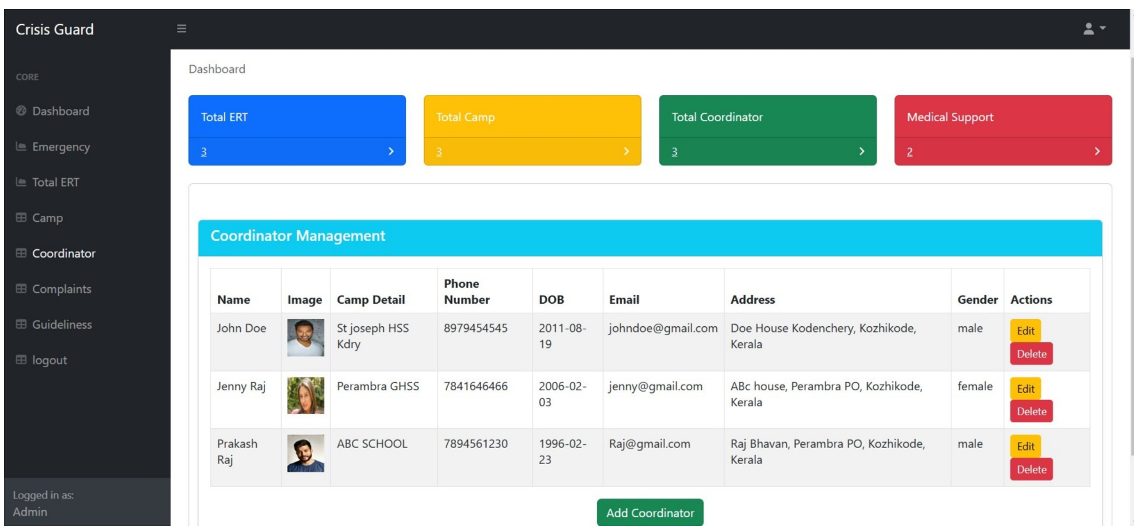
After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specific format. The output generated by the system under considerations is tested by asking the users about the format required by them.

CHAPTER 7

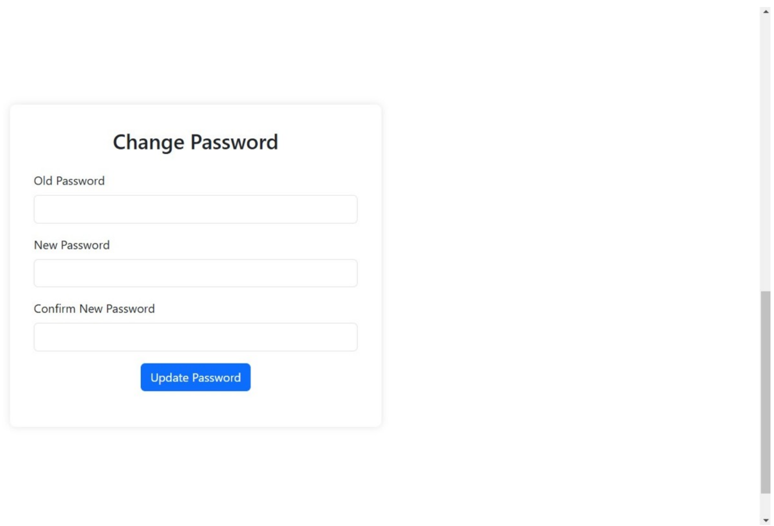
SCREENSHOTS



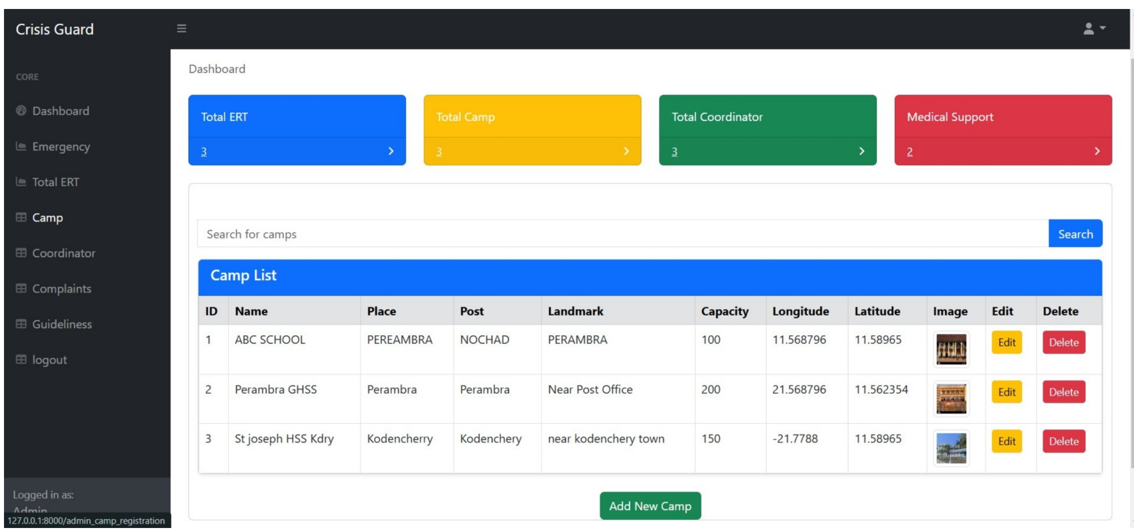
(figure15 Admin dashboard)



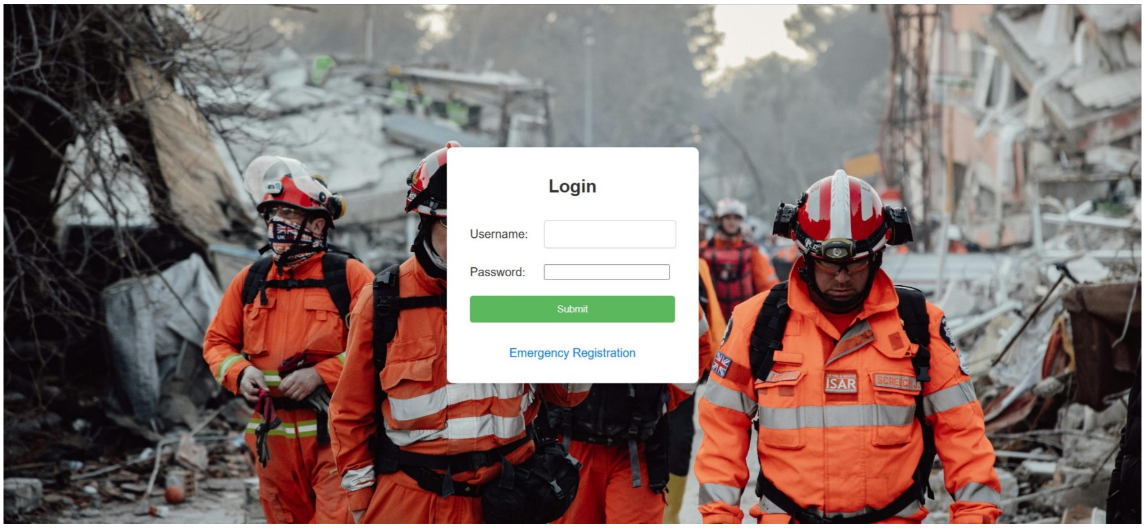
(figure16 Admin coordinator manage)



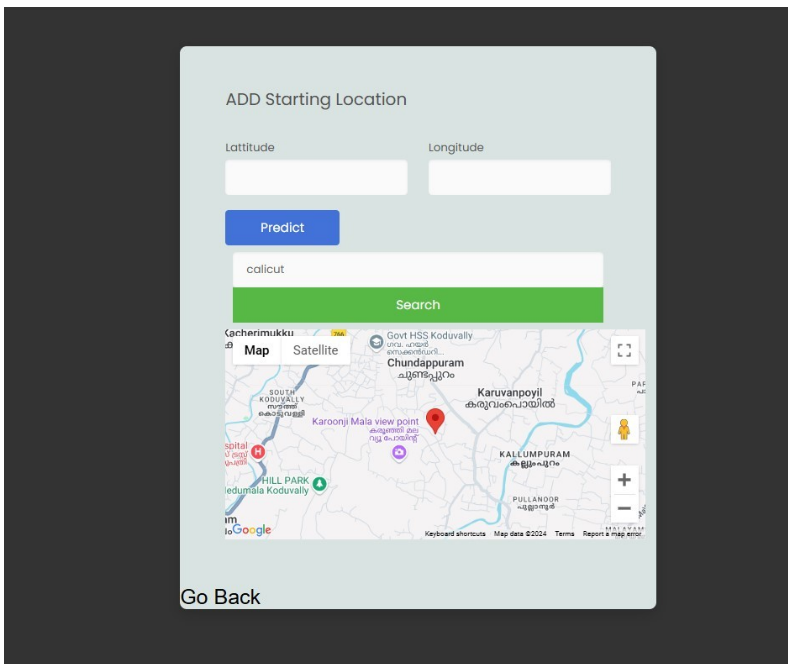
(figure17 change password)



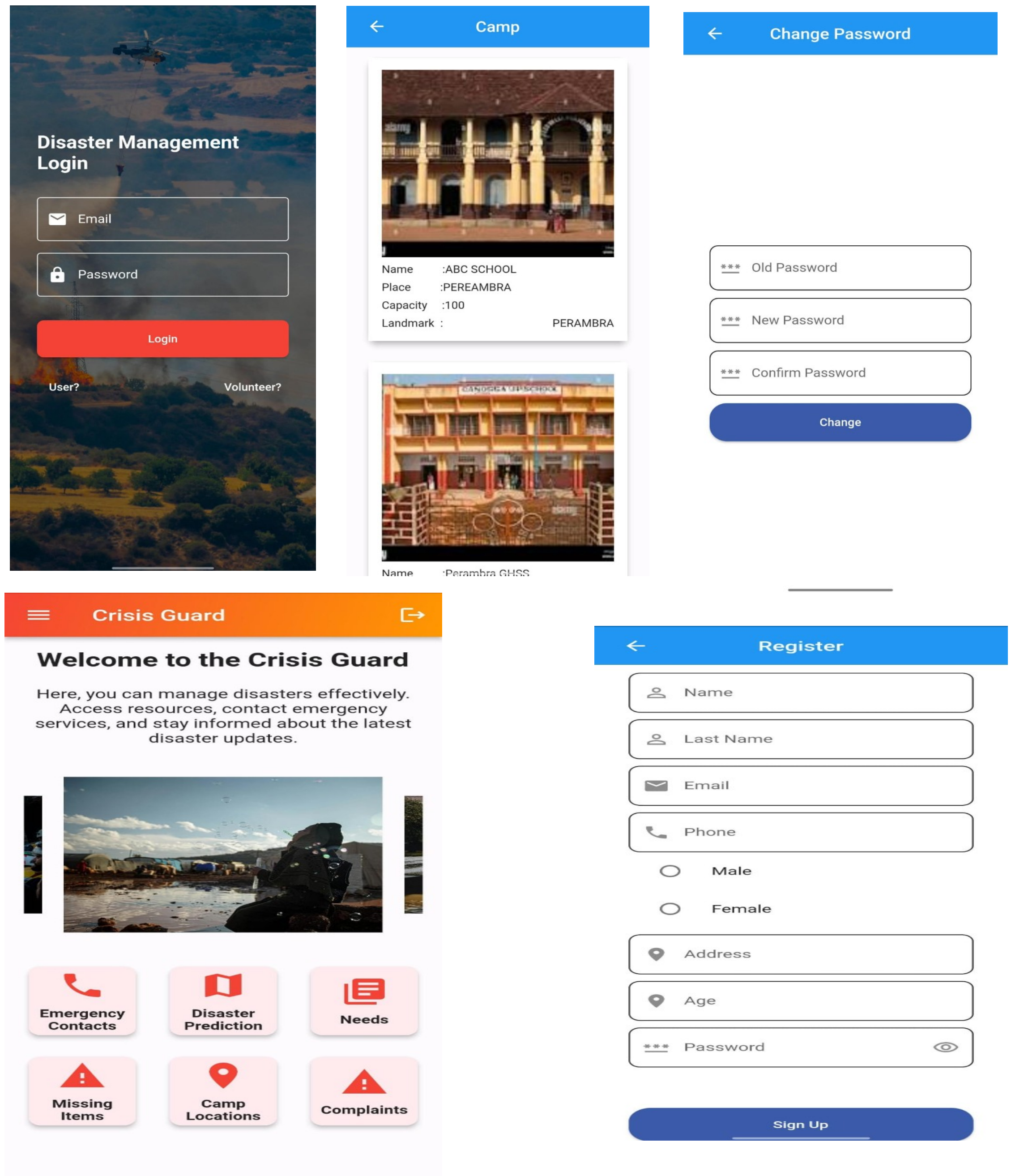
(figure18 CAMP MANAGE)



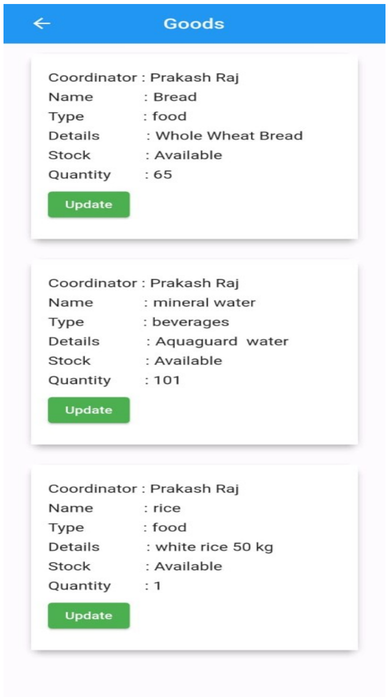
(figure 19 login page)

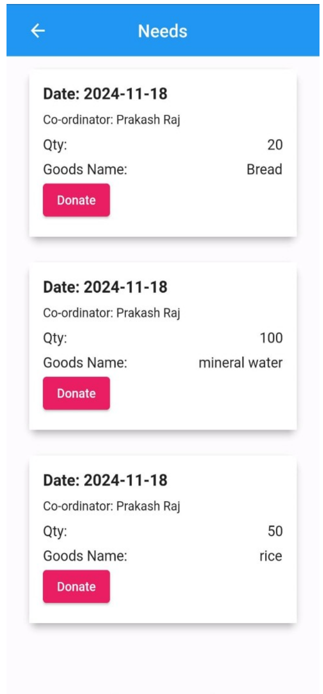


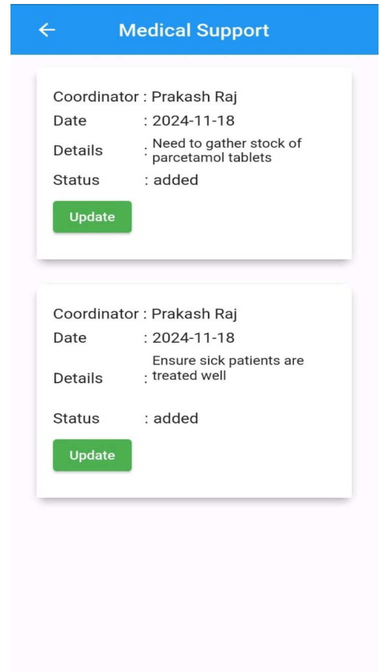
(figure20 PREDICTION)



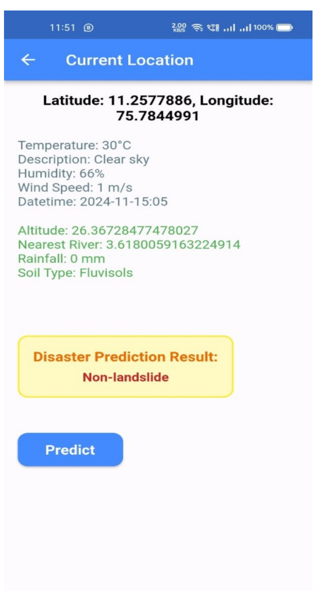
(figure21 USER FEATURES)

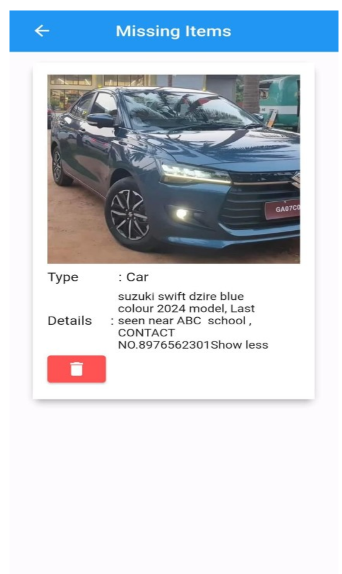






(figure22 STOCK AND MEDICAL SUPPORT)





(figure23 PREDICTION RESULT)

(figure24 MISSING ITEMS)

CHAPTER 8
CONCLUSION

Crisis Guard is a pioneering disaster management system that has the potential to transform the way we prepare for, respond to, and recover from disasters. By integrating advanced weather prediction, emergency response, and relief operations, Crisis Guard enables prompt and effective disaster management. The system's scalability, flexibility, and user-friendly interface make it an indispensable tool for governments, emergency responders, and communities worldwide.

The significance of Crisis Guard cannot be overstated. Natural disasters have become an unfortunate reality in today's world, resulting in devastating losses of life, property, and infrastructure. The importance of effective disaster management has never been more critical. Crisis Guard addresses this critical need by providing a comprehensive and integrated approach to disaster management.

One of the key strengths of Crisis Guard is its ability to facilitate communication and coordination among stakeholders. The system's real-time reporting and updates enable emergency responders to respond quickly and effectively, reducing the risk of further damage and loss of life. Additionally, Crisis Guard's resource management capabilities enable the efficient allocation of resources, ensuring that aid reaches those who need it most.

In conclusion, Crisis Guard is a groundbreaking disaster management system that has the potential to save countless lives and reduce economic and environmental impacts. Its comprehensive and integrated approach to disaster management makes it an essential tool for governments, emergency responders, and communities worldwide.

REFERENCE

-Bollin, C., & Hidajat, R. (2009). Community-Based Disaster Risk Reduction. International Journal of Mass Emergencies and Disasters, 27(1), 20-48.

-B. Fabito , F. Balahadia, and J. D. N. Cabatlao, "AppLERT: A mobile application for Incident and Disaster Notification for Metro Manila", 2016 IEEE Region 10 Symposium (TENSYMP), pp. 288-292, IEEE, 2016.

-McEntire, D. A. (2012). Disaster Response and Recovery: Strategies and Tactics for Resilience. Wiley.

-G. L. Diaz Intal, D. Senoro, and T. Palaoag, "User experience design for disaster management mobile application using design thinking approach", 2020 The 4th International Conference on Software and e-Business, pp. 7-13, 2020.

-M. Stute, M. Maass, T. Schons, M. A. Kaufhold, C. Reuter, and M. Hollick, "Empirical insights for designing information and communication technology for international disaster response. International journal of disaster risk reduction", vol. 47, pp. 101598, 2020.